

Page 1: Product Target and Non-Negotiables

Purpose: Define the product as a premium-feeling phone remote controller where the free edition works brilliantly on local Wi-Fi and the paid edition makes remote access reliable from another network without turning the experience into a laggy slideshow.

Product shape

The product has three client surfaces at launch: browser/PWA, Android app, and iOS app. Desktop app is deliberately postponed, but the Windows host must become clean enough that non-technical users can install it, autostart it, approve devices, and recover from errors.

- Free local Wi-Fi must remain useful without a subscription; paid remote access must buy convenience, relay capacity, account sync, and higher reliability rather than artificially cripple the free version.
- The phone UI must stay centered on the user's original complaint: intuitive mouse control, monitor switching, keyboard, zoom, portrait and landscape usability, and visible cursor tracking.
- The publishable target is not a generic RDP clone; it is a mobile-first controller with a laptop-touchpad mental model and remote video good enough for daily lightweight control.

Technical guardrails

The architecture must separate control input, video transport, pairing/security, device/account management, and packaging. Each piece needs its own tests so parallel Codex instances can work without trampling each other.

- Treat the current Node host, phone PWA, capture page, host console, package scripts, and tests as the baseline product rather than a disposable prototype.
- Freeze a checkpoint before risky work, keep `dist/OpenHostLink.zip` reproducible, and prevent accidental renames like `.disabled-by-incident-response` from reaching GitHub.
- Every workflow must preserve real-input safety gates, session approval, trusted-device behavior, local Wi-Fi launch, and the existing high-quality WebRTC stream until a replacement proves better.

Research signal

The current browser path is valuable but limited: MDN documents that screen capture permission cannot be persisted and requires user activation [S1]. That makes a native host path essential for unattended paid remote sessions.

- Parsec's public technology notes point directly at the performance bar: native control, hardware encode/decode, frame timing, and smooth 60 FPS streaming [S2].
- RustDesk proves a product can be open-source/self-hostable while using rendezvous and relay infrastructure when P2P is not enough [S3].
- Cloudflare and Twilio pricing show why relay traffic must be priced carefully; video bandwidth can destroy margins if every remote session is relayed [S4][S5].

Full build checklist

For Product Target and Non-Negotiables, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Product Target and Non-Negotiables should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Product Target and Non-Negotiables when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding a warning, or claiming that free local Wi-Fi includes everyone can explain what paid remote access adds.
- Reject if a speed improvement is claimed without FPS, latency, or package size evidence. The page argues that native capture/encode work is required for high-speed remote control.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.

- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Product Target and Non-Negotiables, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Product Target and Non-Negotiables', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Product Target and Non-Negotiables, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Product Target and Non-Negotiables requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Product Target and Non-Negotiables into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Product Target and Non-Negotiables is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Product Target and Non-Negotiables, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Page 2: Current Build Baseline and Stabilization

Purpose: Turn the existing working local Wi-Fi controller into a stable baseline that can be moved to GitHub and shared across machines without losing the fragile fixes already discovered.

Repository truth

The current workspace contains the Node host, public phone UI, capture page, host console, packaging scripts, docs, output checkpoints, and tests. Before GitHub migration, the repo must contain normal filenames for `package.json` and `src/server.js`, not disabled copies.

- Run a clean file inventory: root files, `src`, `public`, `packaging`, `scripts`, `test`, `docs`, `dist`, and `output/checkpoints`.
- Move generated heavy artifacts out of the default commit path or place them under release assets; do not commit `node\_modules`, smoke extraction folders, or huge acceptance output.
- Preserve `checkpoint-one-v67-current.zip`, `checkpoint-two-v68-webrtc-current.zip`, and the latest `OpenHostLink.zip` as release/reference artifacts, not active source.

Known working behaviors

The handoff must state the confirmed working pieces: pairing PIN, host approval, trusted sessions, real input, touchpad relative movement, keyboard, high-quality video, capture autostart, package build, and v76 touchpad/zoom fixes.

- Document the exact commands used to run, package, smoke test, and restart the host.
- Document that monitor switching remains intentionally unresolved until the rest of the product is stabilized.
- Document that the browser version is a compatibility layer and native apps must be measured separately.

Stabilization rules

No new feature should land until the baseline has a Git commit, a known branch, a reproducible package, and a smoke test result. This prevents the project from becoming a pile of untraceable fixes.

- Treat the current Node host, phone PWA, capture page, host console, package scripts, and tests as the baseline product rather than a disposable prototype.
- Freeze a checkpoint before risky work, keep `dist/OpenHostLink.zip` reproducible, and prevent accidental renames like `disabled-by-incident-response` from reaching GitHub.
- Every workstream must preserve real-input safety gates, session approval, trusted-device behavior, local Wi-Fi launch, and the existing high-quality WebRTC stream until a replacement proves better.

Full build checklist

For Current Build Baseline and Stabilization, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Current Build Baseline and Stabilization should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Current Build Baseline and Stabilization when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Current Build Baseline and Stabilization, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Current Build Baseline and Stabilization', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Current Build Baseline and Stabilization, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Current Build Baseline and Stabilization requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Current Build Baseline and Stabilization into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Current Build Baseline and Stabilization is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Current Build Baseline and Stabilization, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Workspace filenames are normal and runnable.

Baseline limitations are written down without hiding them.

The next Codex instance can run the project from the document alone.

Page 3: GitHub Migration and Repo Hygiene

Purpose: Move the project into GitHub in a way that supports multiple Codex instances, avoids accidental artifact commits, and gives the user a rollback path.

Repository creation

Create a private GitHub repository first, then decide later whether to open-source the free local Wi-Fi edition. The initial repo should be private because the code still contains rough packaging, experimental capture paths, and host-key assumptions.

- Initialize Git only after `.gitignore` excludes `node_modules`, `dist`, output smoke folders, log files, `.pid`, personal host keys, and local data.
- Commit source, docs, tests, package manifests, scripts, and packaging templates.
- Tag the imported baseline as `v0.1-local-wifi-baseline-v76` or similar so future experiments can be compared cleanly.

Branch model

Use short-lived feature branches mapped to plan chapters. Each Codex instance owns one branch and one workstream; merges happen through PRs with a checklist.

- Example branches: `stabilize/browser-pwa`, `mobile/android-shell`, `mobile/ios-shell`, `infra/remote-relay`, `host/native-capture`, `qa/performance-harness`.
- PRs must include test commands, screenshots for UI changes, and a rollback note.
- Use GitHub Issues as the single list of product tasks, not chat fragments.

Secrets and data

Never commit host keys, pairing tokens, local device trust stores, logs with IP addresses, or screen-capture evidence containing personal content.

- Add a `docs/security/dev-secrets.md` explaining where local secrets live.
- Add sample env files only with fake values.
- Add a pre-commit or CI grep for obvious secrets once the repo is stable.

Full build checklist

For GitHub Migration and Repo Hygiene, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for GitHub Migration and Repo Hygiene should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on GitHub Migration and Repo Hygiene when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For GitHub Migration and Repo Hygiene, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'GitHub Migration and Repo Hygiene', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near GitHub Migration and Repo Hygiene, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing GitHub Migration and Repo Hygiene requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert GitHub Migration and Repo Hygiene into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for GitHub Migration and Repo Hygiene is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up GitHub Migration and Repo Hygiene, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

A GitHub-ready file inclusion/exclusion policy exists.
Parallel branch names are defined.

Release artifacts are separated from source.

Page 4: Handoff to Other Codex Instances

Purpose: Prepare the project so new Codex instances can contribute without needing the entire chat history or guessing what was tested.

Handoff packet

Each new instance receives the handoff document, this 25-page plan, the current repo URL, the branch it owns, the exact task boundaries, and the commands it is allowed to run.

- Include current architecture, known issues, user preferences, setup commands, testing commands, and product priorities.
- Include the rule: do not rewrite shared protocol, auth, or package scripts unless your chapter explicitly owns them.
- Include a required final report format: changed files, tests run, screenshots/artifacts, risks, and follow-up issues.

Collision avoidance

Split ownership by directories and interfaces. One instance can own `public/app.js` zoom UI while another owns `src/rtc-room.js`, but nobody should edit both sides of a protocol without an agreed schema issue.

- Maintain a `docs/parallel-work-ledger.md` with active branches, owner, files, status, and merge order.
- Use interface docs for WebSocket messages, RTC signaling, settings schema, and package launch contracts.
- Keep a merge captain role that reviews PRs and resolves conflicts in a controlled order.

Continuity prompt

The handoff prompt must be operational, not motivational. It should say what exists, what to protect, what to ignore, how to run it, and how to report back.

- Avoid asking new instances to `make it better` broadly.
- Give each instance one acceptance gate and one directory boundary.
- Require them to update docs when they change behavior.

Full build checklist

For Handoff to Other Codex Instances, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Handoff to Other Codex Instances should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Handoff to Other Codex Instances when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped frame, RTT, relay, direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Handoff to Other Codex Instances, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after `Handoff to Other Codex Instances`, link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Handoff to Other Codex Instances, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Handoff to Other Codex Instances requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Handoff to Other Codex Instances into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Handoff to Other Codex Instances is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Handoff to Other Codex Instances, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

A new Codex instance can start without this chat.

File ownership is explicit.

merge/report rules are clear enough to prevent collateral damage.

Page 5: Architecture Split: Free Local vs Paid Remote

Purpose: Define the product architecture around two modes that share UX and host code but differ in network path, account services, and support expectations.

Free local edition

The free edition should run host and phone on the same LAN, plus optional personal Tailscale for technical users. It can be open-source and GitHub-downloadable.

- Use LAN discovery/QR/manual URL and keep Tailscale as a documented free different-network workaround for non-commercial/power users [S6].
- Do not require account login for local mode; pairing PIN and host approval are enough.
- Keep browser/PWA support because it lowers friction and supports quick testing.

Paid remote edition

The paid edition should provide account login, device registry, signaling, NAT traversal, relay fallback, better onboarding, persistent device pairing, and support.

- Prefer P2P direct connections first; use TURN relay only when NAT/firewalls block direct traffic.
- Meter or cap relay bandwidth by subscription tier because 1080p60 video can burn GB quickly.
- Use managed TURN initially for launch speed, then evaluate self-hosted relay when traffic volume justifies it.

Shared core

Control protocol, input safety, monitor metadata, device trust, UI language, and support diagnostics should be shared across free and paid editions.

- Avoid forking the UI into unrelated products.
- Use capability flags from the host/account service to show remote-only features.
- Keep the local edition as a credibility engine and debugging fallback.

Full build checklist

For Architecture Split: Free Local vs Paid Remote, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Architecture Split: Free Local vs Paid Remote should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Architecture Split: Free Local vs Paid Remote when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped frame, RTT, relay-direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Architecture Split: Free Local vs Paid Remote, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Architecture Split: Free Local vs Paid Remote', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Architecture Split: Free Local vs Paid Remote, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Architecture Split: Free Local vs Paid Remote requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Architecture Split: Free Local vs Paid Remote into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Architecture Split: Free Local vs Paid Remote is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Architecture Split: Free Local vs Paid Remote, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

- Free and paid boundaries are explicit.
- No artificial free-version breakage is required.
- Remote costs are designed into the subscription model.

Page 6: Host Capture and Video Pipeline

Purpose: Replace the fragile browser capture dependency with a host capture/encoder pipeline capable of smooth remote video while keeping the browser capture path as fallback.

Current limitation

The existing high-quality stream depends on a Chrome capture window and WebRTC. It can work well, but browser screen sharing cannot be fully unattended in the general case because browser permission is user-mediated [S1].

- Keep the current capture page for local free mode and fallback.
- Document that auto-select flags are browser-specific and may break across machines.
- Do not build a business promise on browser capture alone.

Native capture direction

The host should evolve toward a native capture module using Windows Graphics Capture or Desktop Duplication, then hardware encode through Media Foundation/NVENC/AMF/QuickSync where available.

- Start with Windows because that is the current host target.
- Expose a clean stream interface so later macOS/Linux host support can be added without rewriting phone clients.
- Track capture FPS, encode latency, dropped frames, resolution, cursor composition, and monitor ID.

Performance bar

Parsec's public notes set the engineering target: native low-level control, zero-copy GPU path, hardware decode, and frame-timing optimization [S2]. The paid tier must move in that direction.

- Measure capture, encode, network, decode, draw, and input acknowledgement separately; a single 'laggy' label is not actionable enough for product work.
- Favor native hardware paths for the paid high-speed tier: desktop capture APIs, GPU encode, H.264/H.265/AV1 where supported, hardware decode on Android/iOS, and frame pacing.
- Keep the browser/PWA as a free and universal compatibility layer, but do not pretend it can match native latency when browser capture permission, canvas drawing, and mobile Safari limits apply.

Full build checklist

For Host Capture and Video Pipeline, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Host Capture and Video Pipeline should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Host Capture and Video Pipeline when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package, or device that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding a message, or removing a state without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Host Capture and Video Pipeline, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Host Capture and Video Pipeline', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Host Capture and Video Pipeline, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Host Capture and Video Pipeline requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Host Capture and Video Pipeline into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Host Capture and Video Pipeline is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Host Capture and Video Pipeline, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Browser capture remains fallback, not the final paid path.

Native capture module is specified

Performance metrics are defined before implementation.

Page 7: Remote Networking, NAT Traversal, and Relay

Purpose: Design remote access so most sessions are direct and fast, with relay as a paid fallback rather than the default path.

Connection setup

Use an account-backed signaling service: host registers online status, phone requests a session, host approves/trusts device, then both sides exchange WebRTC offers, ICE candidates, and capability metadata.

- Use STUN for direct UDP discovery.
- Use TURN only after direct candidates fail or are predicted to fail.
- Keep a WebSocket control fallback only for low-bandwidth command signaling, not video.

Relay choices

Cloudflare Realtime TURN is currently attractive for early launch because it has global infrastructure, \$0.05/GB pricing, free STUN, and a 1,000 GB free tier [S4]. Twilio is mature but much more expensive for TURN egress in many regions [S5].

- Prototype with Cloudflare TURN and cost dashboards.
- Add provider abstraction so TURN vendor can be swapped.
- If usage grows, evaluate self-hosted relay nodes like RustDesk-style rendezvous/relay [S3].

Remote safety

Remote access increases abuse risk, so the host must show active sessions, allow one-click stop, require explicit trust for unattended use, and log remote connections clearly.

- Keep all remote control behind explicit pairing, host approval or trusted-device renewal, short-lived tokens, origin checks, command sequence validation, and a host-side kill switch.
- Design the paid remote tier as account-authenticated signaling plus P2P first, TURN relay fallback second, and no permanent public inbound port on a home laptop.
- Log security events without leaking typed text, clipboard contents, full tokens, or private screen content.

Full build checklist

For Remote Networking, NAT Traversal, and Relay, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Remote Networking, NAT Traversal, and Relay should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Remote Networking, NAT Traversal, and Relay when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay-avoided, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Remote Networking, NAT Traversal, and Relay, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Remote Networking, NAT Traversal, and Relay', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Remote Networking, NAT Traversal, and Relay, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Remote Networking, NAT Traversal, and Relay requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Remote Networking, NAT Traversal, and Relay into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Remote Networking, NAT Traversal, and Relay is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Remote Networking, NAT Traversal, and Relay, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

- Remote path is P2P-first.
- TURN cost is measured and priced.
- The relay provider can be changed later.

Page 8: Browser/PWA Version Repair

Purpose: Fix the browser version as a stable free controller surface on mobile and desktop, while being honest about browser capture limitations.

Mobile browser

The PWA must be clean on iPhone Safari, Android Chrome, portrait, landscape, notch/safe-area, and home-screen installed mode. Its controls should not rely on hover, desktop-only keyboard shortcuts, or exact viewport height.

- Retest keyboard toggle, touchpad relative movement, tap/double-tap/hold, two-finger scroll, zoom lens, display sheet, settings sheet, and reconnect.
- Keep touchpad visual state separate from real cursor input forever; the v76 fix must become a regression test.
- Add a mobile browser compatibility matrix to docs.

Desktop browser

Desktop browser support should exist for testing and emergency control, but desktop users are not the primary UX. It needs a sensible layout, keyboard shortcuts, pointer lock where allowed, and no mobile-only assumptions.

- Add responsive desktop styles.
- Use the same pairing/session model.
- Avoid exposing host secrets in desktop test pages.

Capture limits

Browser capture can prompt, require activation, and refuse persistent permission [S1]. The browser edition can display and control a stream, but final unattended host capture should move native.

- Keep capture status visible.
- Explain when a user must touch the laptop.
- Use native path for paid remote promise.

Full build checklist

For Browser/PWA Version Repair, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Browser/PWA Version Repair should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Browser/PWA Version Repair when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Browser/PWA Version Repair, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Browser/PWA Version Repair', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Browser/PWA Version Repair, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Browser/PWA Version Repair requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Browser/PWA Version Repair into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Browser/PWA Version Repair is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Browser/PWA Version Repair, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

- Mobile and desktop browser test matrix exists.
- Known browser capture limits are documented.
- v76 touchpad/zoom regressions stay covered.

Page 9: Android App Plan

Purpose: Build an Android app that starts as a polished WebView/native shell around the existing controller, then progressively replaces performance-sensitive pieces with native code.

Phase 1 shell

Use Kotlin/Android or React Native/Flutter only after choosing the long-term cross-platform strategy. The quickest path is likely a native Android shell that embeds the PWA, manages device identity, push/reconnect, safe-area, haptics, and secure storage.

- Store tokens in Android Keystore-backed storage.
- Use native haptics and keyboard integration.
- Expose share/open-link handling for local LAN URLs and paid account login.

Phase 2 media

Move video receive/decode into native WebRTC where needed. Browser WebView may not provide the same hardware decode and timing control as native WebRTC.

- Use native WebRTC stats for jitter, decode, dropped frames, and RTT.
- Keep the touchpad UI native or hybrid only if gesture fidelity is perfect.
- Build an internal latency overlay.

Play Store readiness

Remote-control apps must present a clear user-owned host model, privacy disclosures, account deletion path, and no deceptive accessibility usage.

- Do not request Android Accessibility permissions for controlling the phone; this app controls the remote computer.
- Declare network/device data collection accurately.
- Include a demo/practice mode for review if the host app is required.

Full build checklist

For Android App Plan, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Android App Plan should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Android App Plan when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package, command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Android App Plan, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Android App Plan', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.

- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Android App Plan, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Android App Plan requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Android App Plan into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Android App Plan is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Android App Plan, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

- Android MVP scope is split into shell and native media phases.
- Secure storage and review needs are included.
- Secure identity remains the product differentiator.

Page 10: iOS App Plan

Purpose: Build an iPhone app that is App Store-compliant, mobile-first, and honest about remote desktop scope.

Apple constraints

Apple's guideline 4.2.7 allows remote desktop clients under specific conditions when mirroring user-owned host devices and not acting as a store-like interface [S7]. The iOS app must be a generic remote desktop/control client, not a way to run or buy hosted software.

- All account management and subscriptions need to follow App Store rules when sold in-app.
- Do not mimic iOS or App Store UI inside the remote stream.

- Include privacy disclosures for device identifiers, diagnostics, and relay usage.

MVP implementation

Start with SwiftUI plus WKWebView only if the PWA UX is already stable, then evaluate native WebRTC and native gestures for performance.

- Use Keychain for tokens.
- Use native haptics, safe-area handling, keyboard accessory controls, and orientation-specific layouts.
- Provide a demo mode or guided screenshots for App Review because the app depends on a host.

Native media path

For paid remote speed, plan native WebRTC receive/decode and a custom gesture layer. The browser path remains a bootstrap, not the final App Store experience.

- Measure capture, encode, network, decode, draw, and input acknowledgement separately; a single 'laggy' label is not actionable enough for product work.
- Favor native hardware paths for the paid high-speed tier: desktop capture APIs, GPU encode, H.264/H.265/AV1 where supported, hardware decode on Android/iOS, and frame pacing.
- Keep the browser/PWA as a free and universal compatibility layer, but do not pretend it can match native latency when browser capture permission, canvas drawing, and mobile Safari limits apply.

Full build checklist

For iOS App Plan, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for iOS App Plan should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on iOS App Plan when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For iOS App Plan, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'iOS App Plan', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near iOS App Plan, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing iOS App Plan requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert iOS App Plan into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for iOS App Plan is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up iOS App Plan, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page
IOS review constraints are reflected in product design.
MVP and native media phases are separated.
Store/privacy requirements are not left until launch week.

Page 11: Input, Gesture, and UX Finalization

Purpose: Make the controls feel intentional rather than experimental, across all client surfaces.

Touchpad model

The primary mobile control model should be touchpad-relative movement, single tap left click, double tap right click, hold/drag gestures, two-finger scrolling, edge-hold movement, and optional direct screen touch.

- The visual touchpad dot must never represent the real cursor; it is a local joystick/hint only.
- Add sensitivity presets for fine, normal, fast, and presentation modes.
- Add tutorial overlays only during onboarding, not as permanent clutter.

Zoom model

Zoom should have two separate concepts: view zoom that crops the whole display, and cursor lens that follows the mouse. Settings must control lens size, lens zoom, hold-to-show, and auto-follow.

- Lens cursor is a small gold dot centered in the lens.
- When auto-follow is enabled, pan toward cursor smoothly and clamp only at real screen edges.
- When zoom is disabled, remove lens and reset view cleanly.

Keyboard and shortcuts

The keyboard button should summon the native keyboard on mobile, while shortcut buttons handle Windows, Ctrl hold, Codex, Claude, and user-defined app launches.

- Shortcuts belong in a configurable rail or sheet.
- Never make the rail so wide it steals landscape control space.
- All shortcuts must fail visibly if the host app cannot launch them.

Full build checklist

For Input, Gesture, and UX Finalization, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Input, Gesture, and UX Finalization should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Input, Gesture, and UX Finalization when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Input, Gesture, and UX Finalization, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Input, Gesture, and UX Finalization', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.

- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Input, Gesture, and UX Finalization, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Input, Gesture, and UX Finalization requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Input, Gesture, and UX Finalization into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Input, Gesture, and UX Finalization is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Input, Gesture, and UX Finalization, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Touchpad, zoom, and keyboard are defined as separate systems. The visual dot cannot move the real cursor. Shortcut UX is configurable instead of hard-coded forever.

Page 12: Monitor Switching and Multi-Display Accuracy

Purpose: Fix monitor switching with explicit identity, source lock, and calibration instead of guessing from fragile browser source order.
Current issue

The known unresolved problem is that switching to another monitor can briefly show the correct screen, then black out or return to screen one. This is likely source-selection/capture identity drift, especially across machines.

- Do not build more features on top of broken monitor identity.
- Record monitor bounds, logical bounds, scale factor, display name, adapter ID where available, and capture source ID.
- Keep manual source lock as an emergency fallback but make auto-detect better.

Native path

Native capture should remove most browser source ambiguity by selecting a monitor through OS APIs rather than Chrome's picker labels.

- For Windows, enumerate displays and capture item IDs.
- Map input coordinates using physical and logical bounds.
- Persist per-machine monitor calibration keyed by stable display signature.

Testing

Build a fake multi-monitor harness plus real physical acceptance checklist.

- Test side-by-side, stacked, mixed DPI, portrait, identical resolution, laptop + external, and monitor disconnect/reconnect.
- Verify video source, cursor overlay, direct touch, touchpad, zoom lens, and screenshot fallback all agree.
- Add a manual 'this screen is wrong' report button that exports source diagnostics.

Full build checklist

For Monitor Switching and Multi-Display Accuracy, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Monitor Switching and Multi-Display Accuracy should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Monitor Switching and Multi-Display Accuracy when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Monitor Switching and Multi-Display Accuracy, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Monitor Switching and Multi-Display Accuracy', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Monitor Switching and Multi-Display Accuracy, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Monitor Switching and Multi-Display Accuracy requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Monitor Switching and Multi-Display Accuracy into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Monitor Switching and Multi-Display Accuracy is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Monitor Switching and Multi-Display Accuracy, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Monitor switching is treated as a system, not a button bug.
Native capture is the preferred fix path.

Mixed-DPI/multi-monitor tests are specified.

Page 13: Security, Trust, and Abuse Prevention

Purpose: Build security as a product feature because remote desktop tools are high-risk by nature.
Threat model

Assets include screen contents, keyboard/mouse control, account identity, device trust, relay credentials, host logs, and billing state. Attackers include wrong-phone local users, stolen tokens, malicious relay clients, and social-engineering misuse.

- Create a formal threat model before paid remote beta.
- Separate pairing PIN, session token, trusted device credential, and account identity.
- Rotate or revoke every credential type.

Host control

The host must always have visible state and control: active sessions, last device, remote IP/network path, streaming status, real input toggle, release all buttons, revoke trust, and stop server.

- Keep all remote control behind explicit pairing, host approval or trusted-device renewal, short-lived tokens, origin checks, command sequence validation, and a host-side kill switch.
- Design the paid remote tier as account-authenticated signaling plus P2P first, TURN relay fallback second, and no permanent public inbound port on a home laptop.
- Log security events without leaking typed text, clipboard contents, full tokens, or private screen content.

Compliance posture

Privacy policy, data inventory, data deletion, store disclosures, and abuse-report pathway must exist before app review or paid launch.

- Never log raw typed text or screen frames by default.
- Make diagnostics opt-in for support exports.
- Keep subscription billing separate from host control permission.

Full build checklist

For Security, Trust, and Abuse Prevention, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Security, Trust, and Abuse Prevention should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Security, Trust, and Abuse Prevention when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Security, Trust, and Abuse Prevention, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Security, Trust, and Abuse Prevention', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Security, Trust, and Abuse Prevention, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Security, Trust, and Abuse Prevention requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Security, Trust, and Abuse Prevention into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Security, Trust, and Abuse Prevention is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Security, Trust, and Abuse Prevention, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Page 14: Account, Billing, and Subscription Design

Purpose: Design the paid tier around real cost drivers and user value instead of vague subscriptions.
Subscription value

Paid remotest access should include remote signaling, TURN relay fallback, device registry, account sync, trusted device recovery, priority capture path, and support diagnostics.

- Free local Wi-Fi stays free.
- Tailscale remains documented for technical users where licensing permits.
- Paid removes setup friction and improves reliability, not basic local control.

Cost model

Relay cost is the main variable. Cloudflare's \$0.05/GB and 1,000 GB free tier are attractive for early testing [S4], while Twilio's regional TURN prices starting around \$0.400/GB show how expensive the wrong provider can be [S5].

- Estimate 720p30, 1080p30, 1080p60, and idle bandwidth.
- Set fair-use limits or quality caps by plan.
- Prefer direct P2P and reduce relay usage through NAT traversal quality.

Billing implementation

Use Stripe for web billing initially if store rules allow; for iOS digital subscription features, evaluate Apple IAP obligations carefully.

- Keep entitlement checks server-side.
- Let users see relay usage and device count.
- Do not block emergency local access because billing failed.

Full build checklist

For Account, Billing, and Subscription Design, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Account, Billing, and Subscription Design should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Account, Billing, and Subscription Design when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding device state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Account, Billing, and Subscription Design, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Account, Billing, and Subscription Design', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.

- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Account, Billing, and Subscription Design, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Account, Billing, and Subscription Design requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Account, Billing, and Subscription Design into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Account, Billing, and Subscription Design is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Account, Billing, and Subscription Design, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Paid value maps to costs.

Relay usage is priced/capped.

Billing rules are separated for web, Android, and iOS.

Page 15: Infrastructure and Backend Services

Purpose: Create the minimum backend that enables paid remote access without overbuilding enterprise infrastructure.

Services

Initial backend services: account auth, device registry, signaling WebSocket, ephemeral TURN credential minting, subscription entitlement, telemetry ingestion, and support log upload.

- Use a managed database and auth provider only if it reduces launch risk.
- Keep signaling stateless where possible.
- Store only metadata needed for operation and support.

Deployment

Start with a simple cloud deployment that supports WebSockets globally enough for beta; then add regional routing if latency data proves it is needed.

- Separate production, staging, and local dev.
- Use IaC once the shape stabilizes.
- Add uptime checks for signaling and TURN credential endpoints.

Observability

Track connection setup time, direct vs relay percentage, TURN GB, input RTT, video FPS, dropped frames, session disconnect reasons, and capture errors.

- Do not capture screen contents.
- Make support bundles user-initiated.
- Build dashboards before charging broadly.

Full build checklist

For Infrastructure and Backend Services, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Infrastructure and Backend Services should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Infrastructure and Backend Services when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality, without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Infrastructure and Backend Services, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Infrastructure and Backend Services', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.

- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Infrastructure and Backend Services, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Infrastructure and Backend Services requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Infrastructure and Backend Services into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Infrastructure and Backend Services is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Infrastructure and Backend Services, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

- Backend services are scoped to paid remote access.
- Telemetry is useful but privacy-preserving.
- Deployment complexity grows only when metrics demand it.

Page 16: Performance Measurement and Optimization

Purpose: Make speed measurable so optimization work is not guesswork.
Latency budget

Break glass-to-glass delay into capture, encode, network, jitter buffer, decode, draw, and input feedback. Set target budgets for local Wi-Fi and paid remote.

- Local target: feel instant for mouse and text, high-quality stream at smooth FPS.
- Remote target: P2P as close to local as network allows, relay acceptable but transparent.
- Record device class because old phones decode differently.

Optimization path

Prioritize the largest delays first: browser capture permission and source drift, canvas decode/draw hitches, WebRTC frame pacing, mouse jitter, and relay path selection.

- Measure capture, encode, network, decode, draw, and input acknowledgement separately; a single 'laggy' label is not actionable enough for product work.
- Favor native hardware paths for the paid high-speed tier: desktop capture APIs, GPU encode, H.264/H.265/AV1 where supported, hardware decode on Android/iOS, and frame pacing.
- Keep the browser/PWA as a free and universal compatibility layer, but do not pretend it can match native latency when browser capture permission, canvas drawing, and mobile Safari limits apply.

Test harness

Build synthetic tests and physical tests: moving clock video, scrolling page, tab dragging, cursor path smoothness, zoom lens follow, monitor switch, and low-bandwidth relay.

- Use automated stats plus screen recordings.
- Make performance reports comparable across commits.
- Block release if FPS or input RTT regresses beyond threshold.

Full build checklist

For Performance Measurement and Optimization, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Performance Measurement and Optimization should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Performance Measurement and Optimization when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Performance Measurement and Optimization, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Performance Measurement and Optimization', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Performance Measurement and Optimization, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Performance Measurement and Optimization requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Performance Measurement and Optimization into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Performance Measurement and Optimization is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Performance Measurement and Optimization, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Latency budget exists.

Optimization targets are measurable.

Release gates include performance regression checks.

Page 17: Packaging, Install, and Autostart

Purpose: Make installation boring for non-technical users while preserving a developer-friendly source build.

Free GitHub download

Provide a GitHub release zip or installer for Windows local Wi-Fi mode. The zip should contain one obvious launcher, instructions, Node runtime strategy, and no confusing stale scripts.

- Decide whether to bundle Node or require installation; for non-technical users, bundled runtime or single installer is better.
- Generate a signed installer later, but keep zip for beta.

- Write `START-HERE.txt` in plain language.

Host autostart

Remote value depends on the host being available when the user is away. Add opt-in startup registration, tray status, health check, and restart-on-crash strategy.

- Do not silently run remote control without visible tray/state.
- Let users disable autostart easily.
- Warn if firewall or capture permission blocks remote use.

Update path

Add versioning, changelog, and migration rules. Do not strand trusted devices or settings when upgrading.

- Semver app and protocol.
- Backward-compatible protocol where possible.
- One rollback package per release.

Full build checklist

For Packaging, Install, and Autostart, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Packaging, Install, and Autostart should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Packaging, Install, and Autostart when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding behind state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Packaging, Install, and Autostart, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Packaging, Install, and Autostart', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.

- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Packaging, Install, and Autostart, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Packaging, Install, and Autostart requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Packaging, Install, and Autostart into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Packaging, Install, and Autostart is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Packaging, Install, and Autostart, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Packaging is beginner-friendly, including tests, or reducing functionality. Autostart is opt-in and visible. Update/rollback rules exist.

Page 18: Testing, CI, and Release Gates

Purpose: Create evidence that the product works before public release and before charging users.
Automated tests

Keep existing Node tests and add CI for protocol, input adapter, session store, RTC room, settings, browser UI, package smoke, and security lint.

- CI must run on PRs.
- Tests that require real screen capture can be nightly/manual.
- Package smoke should use a clean extraction directory.

Manual acceptance

Some features need physical devices: iPhone Safari, Android Chrome, Android app, iOS app, multi-monitor Windows, cellular remote path, and weak Wi-Fi.

- Create checklists with screenshots and expected timings.
- Record device, OS, browser/app version, network type, and result.
- Do not mark a release ready without real phone tests.

Release decision

A release candidate needs green tests, no critical security issues, performance within target, install instructions verified by a fresh machine, and known issues listed.

- Publish release notes.
- Attach zip/installer hashes.
- Tag the Git commit.

Full build checklist

For Testing, CI, and Release Gates, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Testing, CI, and Release Gates should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Testing, CI, and Release Gates when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Testing, CI, and Release Gates, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Testing, CI, and Release Gates', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.

- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Testing, CI, and Release Gates, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Testing, CI, and Release Gates requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Testing, CI, and Release Gates into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Testing, CI, and Release Gates is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Testing, CI, and Release Gates, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

CI scope is defined.
Physical-device gates are defined.

Release criteria are explicit.

Page 19: UI Polish and Product Feel

Purpose: Make the interface feel premium, minimal, and reliable rather than like a diagnostics page.
Phone UI

Keep the black/gold premium theme if it remains readable. Controls should be large enough, evenly sized, and limited to what the user needs during actual control.

- Primary rail: Keys, Zoom, Display, Settings, Ctrl/Win/shortcuts as configured.
- Hide raw counters and debug status from normal mode.
- Landscape mode should prioritize screen size and usable controls.

Host UI

The host console should be calm: connection URL, QR, PIN, sessions, capture status, input safety, settings, logs/export. It should not look different between dev and package unless intentionally branded.

- One launch path.
- Clear capture status.
- Clear remote/local network path.

Onboarding

Onboarding must explain: install host, open phone link or app, pair PIN, approve, enable real input, keep signed in, remote access setup.

- Use short steps and visual QR.
- Add troubleshooting only when needed.
- Include practice/demo mode for store review and nervous users.

Full build checklist

For UI Polish and Product Feel, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for UI Polish and Product Feel should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on UI Polish and Product Feel when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RFT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For UI Polish and Product Feel, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'UI Polish and Product Feel', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.

- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near UI Polish and Product Feel, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing UI Polish and Product Feel requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert UI Polish and Product Feel into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for UI Polish and Product Feel is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up UI Polish and Product Feel, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Normal UI hides debug clutter.
Host/package UI are consistent.

Onboarding maps to real setup steps.

Page 20: Documentation and Support System

Purpose: Turn the project's scattered knowledge into durable docs that users and Codex instances can rely on.

Developer docs

Create docs for architecture, protocol, capture lifecycle, input mapping, monitor switching, packaging, tests, and deployment.

- Every shared interface gets a short contract file.
- Every tricky bug gets a regression note.
- Docs should name owners and acceptance gates.

User docs

Create quick-start, troubleshooting, remote access setup, security explanation, and uninstall guides.

- Write for non-technical family users first.
- Keep advanced Tailscale/local firewall steps separate.
- Add screenshots after UI stabilizes.

Support docs

Support needs diagnostic export, version info, network path, relay/direct status, capture stats, and safe redaction.

- No raw screen screenshots unless user explicitly attaches them.
- No raw typed text.
- Make logs understandable enough for issue reports.

Full build checklist

For Documentation and Support System, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Documentation and Support System should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Documentation and Support System when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Documentation and Support System, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Documentation and Support System', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.

- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Documentation and Support System, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Documentation and Support System requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Documentation and Support System into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Documentation and Support System is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Documentation and Support System, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Developer and user docs are separate. Support export is privacy-aware.

Bug knowledge becomes docs/tests.

Page 21: Store, Legal, and Policy Readiness

Purpose: Prepare for app review and payment rules before the apps are submitted.
iOS

Apple's remote desktop rule requires careful positioning as a generic remote desktop client for user-owned host devices [S7]. Avoid store-like browsing inside the remote stream and prepare a demo account/mode.

- Privacy labels must cover diagnostics, identifiers, account info, and crash logs.
- Subscription/IAP policy must be reviewed before selling remote access inside iOS.
- App review notes must explain host setup clearly.

Android

Prepare Play Store data safety, privacy policy, account deletion, and no unnecessary sensitive permissions.

- Do not request microphone unless laptop-output audio architecture actually needs it and user chooses it.
- Do not misuse accessibility services.
- Use foreground service notifications if background connectivity is needed.

Legal

Remote desktop can be abused, so terms must forbid unauthorized control, require user-owned devices, and explain logs, relay, and security.

- Add privacy policy.
- Add terms of service.
- Add responsible-use language.

Full build checklist

For Store, Legal, and Policy Readiness, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Store, Legal, and Policy Readiness should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Store, Legal, and Policy Readiness when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package command, or physical-device check that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding debug state, or reducing functionality without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, R/T, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Store, Legal, and Policy Readiness, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Store, Legal, and Policy Readiness', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.

- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Store, Legal, and Policy Readiness, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Store, Legal, and Policy Readiness requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Store, Legal, and Policy Readiness into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Store, Legal, and Policy Readiness is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Store, Legal, and Policy Readiness, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Store policy risks are known early.
Privacy/legal docs are on the roadmap.

App review demo strategy is included.

Page 22: Parallel Workstream Map

Purpose: Divide work so multiple Codex instances can move quickly without clobbering one another.
Chapter ownership

Assign separate chapters to separate branches. The merge captain updates shared docs and resolves protocol changes.

- Workstream A: repo hygiene, CI, GitHub migration.
- Workstream B: browser/PWA polish and regression tests.
- Workstream C: Android shell prototype.
- Workstream D: iOS shell prototype.
- Workstream E: native host capture research/spike.
- Workstream F: remote signaling/TURN backend prototype.

Interface locks

Shared interfaces need mini RFCs before change: WebSocket messages, RTC signaling, auth/session schema, settings storage, capture metadata, billing entitlement.

- A worker may add fields compatibly.
- A worker may not rename/remove shared fields without RFC approval.
- Each PR updates tests and docs.

Merge order

Merge stabilization first, then browser fixes, then backend prototypes, then mobile shells, then native capture experiments.

- Keep risky native capture on a branch until it beats current stream.
- Keep paid backend behind feature flags.
- Never block local free mode on paid services.

Full build checklist

For Parallel Workstream Map, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Parallel Workstream Map should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Parallel Workstream Map when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package, and device used to reproduce the change that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding state, or removing a control without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Parallel Workstream Map, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Parallel Workstream Map', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Parallel Workstream Map, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Parallel Workstream Map requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Parallel Workstream Map into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Parallel Workstream Map is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Parallel Workstream Map, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

Workstreams have owners and branch names.

Shared interfaces are documented.

Merge order avoids collateral damage.

Page 23: 90-Day Execution Roadmap

Purpose: Sequence the work from handoff to beta without pretending everything can be done at once.
Days 1-14

Stabilize repo, GitHub, CI, handoff docs, package, browser regression tests, and issue tracker. No paid backend work should distract from making the baseline reproducible.

- Create private repo and baseline tag.
- Fix package naming and one-launcher confusion.
- Write architecture and protocol docs.
- Run full test suite and package smoke on a clean PC.

Days 15-45

Build browser/PWA polish, Android shell, iOS shell, remote signaling prototype, and native capture spike in parallel branches.

- Measure current latency.
- Prototype Cloudflare TURN credentials.
- Prototype app shells with real pairing.
- Prototype Windows native capture feasibility.

Days 46-90

Choose the high-speed path, harden account/subscription backend, run closed beta, polish onboarding, and prepare store submissions.

- Beta with 5-20 trusted testers.
- Collect relay cost data.
- Decide launch pricing.
- Prepare App Store/Play Store review assets.

Full build checklist

For 90-Day Execution Roadmap, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for 90-Day Execution Roadmap should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on 90-Day Execution Roadmap when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package name, branch, device, and the exact broken or missing behavior that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding a control, or removing a control without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For 90-Day Execution Roadmap, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after '90-Day Execution Roadmap', link this page, and list the exact files or services that are in scope.

- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near 90-Day Execution Roadmap, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing 90-Day Execution Roadmap requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert 90-Day Execution Roadmap into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for 90-Day Execution Roadmap is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up 90-Day Execution Roadmap, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

- Roadmap is phased.
- Pairing stabilization comes first.
- Beta and monetization are tied to measured quality.

Page 24: Later Desktop App and Platform Expansion

Purpose: Keep the desktop client out of the immediate launch scope while still designing today's protocol, account model, and media path so a future desktop viewer is not boxed in.

Why it waits

The user explicitly wants desktop app work later. That is the right call because the current bottleneck is host reliability, mobile control quality, remote networking, and native phone apps.

- Do not split attention into Electron/Tauri desktop clients before mobile and paid remote access are proven.
- Keep the browser desktop view usable for testing, but do not polish it at the expense of phone UX.
- Record desktop-client assumptions now so protocol choices do not block it later.

Protocol readiness

A future desktop viewer should reuse the same signaling, device trust, entitlement, capture metadata, and input command model.

- Avoid mobile-only names in shared protocol fields.
- Keep keyboard/mouse commands expressive enough for desktop clients.
- Keep stream negotiation capability-based instead of hard-coding phone constraints.

Expansion order

After paid remote beta proves speed and economics, evaluate desktop clients, macOS host support, Linux host support, browser extensions, and enterprise management.

- Desktop viewer can become a paid productivity feature later.
- macOS/Linux host support require separate capture/input/security research.
- Enterprise features should wait until consumer reliability is solid.

Full build checklist

For Later Desktop App and Platform Expansion, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Later Desktop App and Platform Expansion should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Later Desktop App and Platform Expansion when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package, or device that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding a message, or changing a setting without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.
- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Later Desktop App and Platform Expansion, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Later Desktop App and Platform Expansion', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.

- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Later Desktop App and Platform Expansion, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Later Desktop App and Platform Expansion requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Later Desktop App and Platform Expansion into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Later Desktop App and Platform Expansion is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Later Desktop App and Platform Expansion, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.

Completion goals for this page

- Desktop app is intentionally deferred.
- Web architecture remains future-compatible.
- Expansion does not distract from mobile-first launch.

Page 25: Acceptance Gates and Goal Ledger

Purpose: Summarize every goal without skipping steps so the plan can become GitHub issues and Codex chapters.

Gate 1: handoff

The project is not ready for parallel work until the handoff document exists, source files are restored, GitHub repo is created, baseline branch/tag exists, and a new Codex instance can run tests from the docs alone.

- Handoff doc delivered.
- Transfer plan to user's PC documented.
- Codex prompt pack included.
- Git ignore and artifact policy defined.

Gate 2: product foundation

The free local product is shippable when the browser/PWA, host console, packaging, tests, and install docs work on a clean Windows machine and at least one iPhone and one Android phone.

- No debug clutter in normal UI.
- Keyboard/touchpad/zoom/display/settings pass manual checks.
- Package zip or installer is clean.
- Known monitor-switching limitation is either fixed or clearly marked before public release.

Gate 3: paid remote

The paid version is viable only after P2P remote sessions work, TURN fallback is measured, relay cost fits pricing, native/mobile performance beats the browser path, security review passes, and closed beta users say it feels smooth enough.

- Cloudflare/Twilio/self-host relay decision documented.
- Subscription entitlement works.
- Store/legal requirements are satisfied.
- Performance dashboard proves remote speed.

Sources

Evidence used in this plan is carried into the handoff manual's research-source appendix. Future instances should use that appendix to verify browser capture limits, low-latency architecture assumptions, TURN pricing, Tailscale positioning, and app-store policy before making launch promises.

- Do not repeat vague research; reopen the primary source when the decision is pricing, platform policy, browser behavior, or paid remote-access architecture.
- Convert each source constraint into a GitHub issue so research becomes work instead of decoration.

Full build checklist

For Acceptance Gates and Goal Ledger, the worker must treat the page as a build manual. The checklist below is deliberately concrete so it can be copied into GitHub issues without losing the operational steps.

- Start by writing the current behavior in one paragraph using real evidence from the workspace, package, browser, phone, server log, or test output.
- List every file that will be read before editing and every file expected to change. If the list grows, stop and update the issue scope before continuing.
- Identify the user-visible behavior, the host-side behavior, the phone-side behavior, the security behavior, and the failure behavior separately.
- Define the fallback path before implementing the new path. The local Wi-Fi controller must remain usable even if a paid, native, mobile, or relay experiment fails.
- Add or update tests that lock the original bug or capability. Where automation cannot cover it, write the physical-device test with exact device, browser/app, network, display layout, and expected result.
- Update the documentation in the same branch, including user-facing instructions if behavior changed and developer notes if a contract, protocol, or launch command changed.
- Package or run from a clean checkout before calling the issue finished. A feature that only works in the warm development session is not done.

Detailed implementation steps

The implementation for Acceptance Gates and Goal Ledger should move through these steps in order. Skipping the early evidence and contract work is how the prototype kept regressing.

- Step 1: capture baseline evidence. Record current version, command used, URL used, package name, branch, device, and the exact broken or missing behavior.
- Step 2: write the interface contract. For UI work, this means visible states and gestures. For host work, this means endpoints, messages, settings, or process behavior. For backend work, this means auth, session, entitlement, and network states.
- Step 3: implement the smallest vertical slice that proves the contract. Avoid sweeping rewrites unless the issue is explicitly a rewrite chapter.
- Step 4: add regression protection. Prefer unit tests for pure logic, browser tests for UI state, package smoke for launch behavior, and physical checklists for phone/capture behavior.
- Step 5: run the exact acceptance gate. If the gate cannot run locally, document the blocker and attach the next-best evidence instead of pretending it passed.
- Step 6: write the handoff note. Include what changed, what did not change, what remains risky, and how to roll back without losing unrelated work.

Rejection criteria

A reviewer should reject work on Acceptance Gates and Goal Ledger when it only looks finished. These rejection cases are designed to prevent a repeat of broken screen capture, jittery controls, stale packages, and machine-specific behavior.

- Reject if the worker cannot name the test command, package, or device that proves the change.
- Reject if a UI change hides a bug by removing a control, hiding a message, or dropping a state without documenting the tradeoff.
- Reject if a speed improvement is claimed without FPS, latency, dropped-frame, RTT, relay/direct, or device evidence.
- Reject if the change works on the development laptop but the branch has no plan for fresh Windows machines, different Chrome installs, different monitor layouts, or phone browsers.

- Reject if a remote-access or trusted-device change weakens pairing, host approval, token expiry, revoke, release-buttons, or stop-control behavior.
- Reject if source, package, docs, and release artifact disagree about the launcher name, host URL, QR code behavior, or setup steps.

Execution sequence

This page becomes work only when it is converted into issues with owners, files, tests, and acceptance evidence. For Acceptance Gates and Goal Ledger, the sequence below is the default order unless a later spike proves a dependency is wrong.

- Open or update a GitHub issue named after 'Acceptance Gates and Goal Ledger', link this page, and list the exact files or services that are in scope.
- Create a short branch from the current baseline; do not mix unrelated cleanup, UI polish, backend experiments, and package changes in one branch.
- Write or update the contract first: expected behavior, data shape, command, UI state, or user-facing promise.
- Implement the smallest complete slice that can be tested end to end, then add regression coverage before expanding the slice.
- Finish with proof: command output, package smoke, browser/mobile screenshot, performance numbers, or physical-device acceptance notes as appropriate.

Parallel ownership and collision rules

Parallel Codex instances may work near Acceptance Gates and Goal Ledger, but only if each owns a different boundary and updates the work ledger before editing.

- UI-only work owns `public/\*` plus matching tests; protocol or server behavior changes require a separate interface note.
- Host/backend work owns `src/\*` or cloud service code and must preserve local free mode unless the issue explicitly changes that contract.
- Mobile app work must not silently fork control semantics; any gesture or command difference needs a compatibility note.
- Packaging/docs work can update instructions and release artifacts, but must not hide known limitations or mark unverified behavior as done.

Failure modes to check before closing

The common failure pattern for this project is a feature looking good on one machine while breaking another tester's phone or monitor setup. Closing Acceptance Gates and Goal Ledger requires checking the failure modes, not just the happy path.

- Fresh machine or clean profile behaves differently from the development laptop.
- Phone Safari, Android Chrome, or app WebView handles touch, keyboard, screen size, or media playback differently.
- Multi-monitor, mixed-DPI, external display, or reconnect path changes the coordinate/capture assumptions.
- A paid-remote change accidentally breaks the free local Wi-Fi path or requires an account where local mode should not.
- A performance fix improves FPS while making input jitter, zoom follow, audio, or battery usage worse.

Issue breakdown for a worker

Convert Acceptance Gates and Goal Ledger into a set of small issues instead of one huge vague assignment. Each issue should have a single mergeable result and a clear rollback point.

- Issue 1: write the exact current-state evidence before changing anything, including file names, commands, screenshots, or API output.
- Issue 2: define the target behavior in one short contract so future code, UI, docs, and tests agree.
- Issue 3: implement the first narrow path and keep the old path available behind a flag or fallback until the new path is proven.
- Issue 4: add regression coverage for the exact bug or capability, including the machine/device condition that originally exposed it.
- Issue 5: update the user-facing documentation and the developer handoff notes so the next instance does not rediscover the same detail.
- Issue 6: package or deploy the change only after the branch can be tested from a clean checkout or clean extraction.

Evidence artifacts to attach

The work for Acceptance Gates and Goal Ledger is not complete just because the code looks plausible. Attach proof that a reviewer can inspect without trusting the worker's memory.

- Command transcript or test output showing the relevant automated tests passed.
- Screenshots or rendered page previews for any user-interface, onboarding, store, or document change.
- Performance numbers for any capture, networking, video, input, relay, or mobile-app claim.
- A short risk note naming what was not tested, especially physical phones, different networks, and multi-monitor setups.
- A rollback note naming the branch, tag, zip, feature flag, or previous implementation to return to if testers report a regression.

Handoff notes for the next instance

When another Codex instance picks up Acceptance Gates and Goal Ledger, it should start from these notes rather than from the chat history.

- Read this page, the handoff manual, and the files named by the issue before editing.
- State the file ownership boundary in the first progress update.
- Do not mark the issue complete unless the acceptance evidence is stronger than a visual guess.
- If a blocker appears, leave a precise reproduction path and continue with adjacent documentation or tests instead of stopping the whole product plan.